

A Deadlock-Free Scheduling Framework for Adaptive Traffic Signal Control in Heterogeneous Traffic Using YOLOv11-Based Vehicle Detection.

Thanuj Kumar Maligi

Department of Artificial Intelligence
Amrita School of Artificial Intelligence, Bengaluru
Amrita Vishwa Vidyapeetham, India
bl.en.u4aid23030@bl.students.amrita.edu

Pooja Gowda

Department of Computer Science and Engineering
Amrita School of Computing, Bengaluru
Amrita Vishwa Vidyapeetham, India
g_pooja@blr.amrita.edu

Abstract—Heterogeneous, non-lane-disciplined traffic in developing economies induces intense urban congestion. Traditional Intelligent Transportation Systems (ITS) rely on fixed-time induction loop controllers, which cannot handle dynamic vehicle distributions. Existing approaches lack the ability to classify small vehicles, e.g., auto-rickshaws, bikes, and cycles, due to which similar vehicles’ movements are constrained under heavy loads, and they lack formal safety mechanisms to prevent intersection gridlocks. This study aims to address the dynamic vehicle distribution classification and scheduling of the vehicle movement effectively using Multi-Level Feedback Queue (MLFQ) scheduling for traffic management, Banker’s deadlock avoidance for safety check, and Priority Inheritance. To dynamically set green splits, the system compares multiple regression methods—ranging from basic linear equations and decision trees to ensemble forests and gradient-boosted trees (XGBoost)—using real-time inputs from a YOLOv11-S detector. The YOLOv11 is pretrained on the Indian Institute of Science (IISc) Bengaluru UVH-26 dataset for real-time heterogeneous vehicle tracking. On evaluation of the model, it was observed that under severe congestion, ST-OS reduces average lane wait times by 20.7% compared to Round-Robin and 14.4% compared to Most Vehicles First (MVF) schedulers. Furthermore, transfer learning resolves three-wheeler invisibility, establishing a deadlock-free, deployable control framework for smart cities.

Index Terms—Intelligent Transportation Systems, YOLOv11, XGBoost, Multi-Level Feedback Queue (MLFQ), Heterogeneous Traffic

I. INTRODUCTION

Urban traffic management at signalised intersections presents a highly dynamic and non-linear resource scheduling problem, particularly in rapidly growing cities like Bengaluru, India. While current deployments utilise Advanced Traffic Control Systems (ATCS) operating under Vehicle Actuated Control (VAC), Manual (MNL), and Fixed-Interval (INL) modes, they suffer from severe structural and operational deficiencies. Especially, these systems struggle to detect and classify region-specific vehicles (such as auto-rickshaws and partially occluded two-wheelers), causing a substantial portion of the traffic flow to remain unaccounted for in signal-control decisions. Furthermore, these systems lack formal avoidance of deadlock mechanisms for handling downstream congestion

and often fail to ensure fair service allocation across approaches, which leads to lane starvation under traffic imbalances. Emergency vehicle prioritization still depends mostly on manual intervention, with the limited support for automatic queue-clearing mechanisms. We resolve these operational bottlenecks by introducing a unified sensing and adaptive signal dispatching framework.

For real-time traffic state estimation, we deploy a computer vision pipeline that replaces traditional synthetic simulators with a live vehicle count and telemetry stream. The computer vision layer utilizes a lightweight YOLOv11-S object detector fine-tuned on the domain-specific IISc Bengaluru UVH-26 dataset to detect and classify regional vehicle classes (e.g., auto-rickshaws and two-wheelers) that are typically omitted or misclassified by standard models. A ByteTrack multi-object tracking algorithm is integrated to maintain persistent vehicle trajectories, and polygon Regions of Interest (ROIs) are defined to measure active queue growth rates and count vehicles. This multi-class telemetry is bridged via a JSON interface to establish a live Sparse-Sensor Digital Twin, mapping real-world physical congestion levels to the virtual control environment.

Once real-time vehicle distributions and Passenger Car Unit (PCU) weights are captured, they are processed through an AI/ML-based green-time optimization layer. We predicted signal timings by training a variety of classic and modern regressors, comparing simple linear models and decision trees against complex forest and gradient boosted models. These models utilize a combination of traffic characteristics, such as the density of the queue, speed of the vehicle, and growth rate of the queue. We estimated ensemble methods such as Random Forest and XGBoost due to their ability to effectively perform in changing traffic conditions with more accuracy than the standard fixed time controls. The predicted green times are then managed by operating system scheduling mechanisms, such as Multi-Level Feedback Queue (MLFQ) and the Banker’s safety algorithm, to ensure fairness while avoiding gridlocks in the intersection.

To handle these problems systematically, this paper pro-

poses the Smart Traffic Operating Systems, a framework that adapts the core operating-system resource-management techniques to the physical management of traffic signals. In our model, we represent a direct structural mapping where the signalized intersection operates as the processor (CPU), lane queues correspond to process queues, arriving vehicles are treated as active performance threads, and the green phase durations correspond to allocated processing windows. By using this correspondence, we implement concepts of operating systems, including Multi-Level Feedback Queue (MLFQ) scheduling, interrupt-based control mechanism (ISR) with Priority Inheritance Protocol, and Banker's algorithm to prevent deadlock. The MLFQ scheduler is divided into four priority levels that implement the real-time traffic conditions. Q0 controls from starvation and the growth of congestion, Q1 uses the Old Process First (OPF) algorithm to send the lane that consists of the longest waiting vehicles. Q2 develops the Most Vehicles First (MVF) mechanism to maximize the clearing of vehicles that have more vehicles. Q3 operates as a Round Robin fallback queue to maintain fairness among all the approaching lanes. In combination, these mechanisms enhance the flow of traffic, preventing the lane from starving, which maintains smooth traffic movement without congestion.

II. LITERATURE SURVEY

The Urban traffic intersections are more complex and challenging, which are in many research areas, including computer vision for the real-time detection of traffic conditions, multi-class traffic estimation, study-state and ML-based traffic predictions, dynamic control in the signals, and smart transportation frameworks. Here, we examine recent literature in these domains, highlighting key achievements, technical limitations, and open areas.

Wang and co-researchers [1] created YOLO-LIGHT, a simplified object detection model tailored for real-time vehicular tracking on low-power devices, using compressed convolutional layers to maintain stability across varying light levels. In a different approach, Li et al. [2] introduced attention modules into the SNCE-YOLO architecture to improve detection accuracy on complex roads; however, their study was limited to structured lanes with minimal mixed traffic tests. For tracking, Liu et al. [3] designed the BIMSA-YOLO architecture with multi-scale feature integration, though its computational weight restricted real-time deployment.

To capture mixed traffic dynamics, Ashok and Chilukuri [4] utilized macroscopic fundamental diagrams (MFDs) to estimate traffic states without categorizing vehicle types, but their spatial averaging method missed localized queue build-ups at intersections. Deshmukh et al. [5] proposed VDnet, which integrated bi-directional feature fusion to resolve vehicle occlusions, though the model's processing footprint was too large for immediate deployment on edge systems. Focusing on local road compositions, Priya et al. [6] designed a YOLO-based classification pipeline to estimate PCU capacities, validating its detection reliability under highly clustered Indian traffic conditions.

Turnip et al. [7] evaluated machine learning regressors and found that random forests managed the non-linear properties of congestion well, though their work was limited to historical data and could not adjust dynamically. Ru et al. [8] combined forecasting models inside VISSIM for signal timing, but the concurrent processing overhead restricted its capability to adapt to unexpected traffic spikes. To predict traffic progression, Dhondt et al. [9] utilized regression methods combined with empirical mode decomposition, but the multi-stage input processing added latency that prevented real-time timing changes. Similarly, Dong et al. [10] used an XGBoost model with wavelet filters to estimate short-term flows, but the computational lag from the filtering pipeline limited its use for sub-second signal adjustments. For smart city infrastructures, Zhou et al. [11] proposed a short-term forecasting system utilizing 5G vehicle networks and edge nodes, though it remained highly dependent on constant connectivity. To maximize arterial throughput, Huang et al. [12] studied cooperative scheduling using actor-critic policy gradients, which suffered from slow learning convergence in dynamic network topologies.

Wang et al. [13] calculated green splits by modeling queue propagation with bilinear systems and adaptive optimization, though their work assumed simplified, lane-bound traffic flow. Feng et al. [14] developed the EVHF-GCN preemption model using graph convolutional networks to prioritize emergency vehicles, but triggering priority signal phases without checking downstream capacity caused localized congestion blocks. To refine signal timings, Yan et al. [15] implemented an iterative optimizer initialized with random forest predictions, though its search iterations remained too computationally expensive for real-time actuation. For dynamic signal coordination, Ravichandran et al. [16] developed an adaptive controller that adjusts green intervals based on lane-specific density estimates from YOLOv8. For emergency vehicles, Shaamili et al. [17] proposed an intelligent wireless-sensor-based signal control framework to preempt intersections and ensure unhampered mobility.

Huang et al. [18] formulated a cooperative multi-agent traffic control framework using a prioritized experience replay Soft Actor-Critic (MPER-SAC) algorithm to optimize training stability and intersection efficiency in IoT networks. While the prioritized replay mechanism increases sample efficiency and reduces average vehicle delay, the model-free controller still requires intensive offline exploration and lacks deterministic safety guarantees to prevent physical junction deadlocks during out-of-distribution traffic surges. Chen et al. [19] leveraged coevolutionary multi-agent reinforcement learning (Coevo-MARL) to model spatial dependencies and coordinate multi-intersection signal control. Despite its flexibility, this deep reinforcement learning architecture requires intensive offline training and lacks deterministic safety guarantees to prevent lane starvation or gridlock. To coordinate signal control across heterogeneous intersections, Dong et al. [20] presented a distributed multi-agent reinforcement learning framework. The proposed approach improves traffic progression along arterial

corridors; however, policy optimization exhibits slow convergence when traffic patterns change dynamically. Furthermore, the framework lacks an explicit strategy for mitigating localized traffic deadlocks.

El Marai et al. [21] experimented on a road infrastructure digital twin by clustering multi-view video streams under bandwidth constraints to enhance situational awareness. However, their framework served as a passive monitor decoupled from physical signal actuators. Hui et al. [22] developed a collaborative optimization scheme that simultaneously addresses individualized routing and global traffic management. However, the concurrent optimization layers introduce prohibitive computational complexity on standard edge systems. Wang et al. [23] designed UniTSA, a generalizable reinforcement learning system that utilizes dynamic parameters and beacon telemetry for V2X signal control. Despite its flexibility, their performance degraded significantly under low connectivity where V2X penetration is sparse. An adaptive edge-computing traffic signal controller was presented by Hassan et al. [24], where YOLO-based vehicle recognition and Passenger Car Equivalent (PCE) weighting are employed to estimate lane density under real-time conditions. However, the reliance on the conventional COCO dataset restricts recognition of locally prevalent vehicle categories, including three-wheelers. Additionally, the heuristic signal scheduling approach lacks formal analytical guarantees to eliminate deadlock scenarios. Zhang et al. [25] presented a digital twin-enabled intelligent traffic control system in which deep reinforcement learning is applied to optimize signal decisions using virtual intersection models. While the framework supports adaptive traffic management, its reliance on large-scale state representations and model-free learning significantly increases offline training requirements. Another limitation is the absence of deterministic safety guarantees to prevent prolonged waiting or queue starvation.

Existing traffic management systems, including recent edge-deployed and digital-twin control frameworks, cannot handle cluttered, heterogeneous traffic because they cannot reliably detect regional vehicle classes or provide analytical bounds for resolving real-time junction deadlocks. We present the Smart Traffic Operating System (ST-OS), an edge-deployable framework that solves these challenges. ST-OS generates real-time digital twins using a YOLOv11 model fine-tuned on the IISc Bengaluru UVH-26 dataset. The extracted multi-class telemetry is processed by various predictive regressors, such as linear models, decision trees, random forests, and XGBoost, to optimize green-light intervals. To ensure fairness and eliminate physical gridlock, ST-OS incorporates classic operating system algorithms: Multi-Level Feedback Queues (MLFQ) for queue management, the Banker's Algorithm for deadlock avoidance, and Priority Inheritance for emergency-vehicle preemption.

III. PROPOSED METHODOLOGY

A. Data

The presented Smart Traffic Operating System (ST-OS) is validated using two data streams: a historical traffic feature log for optimizing green time and multi-class computer

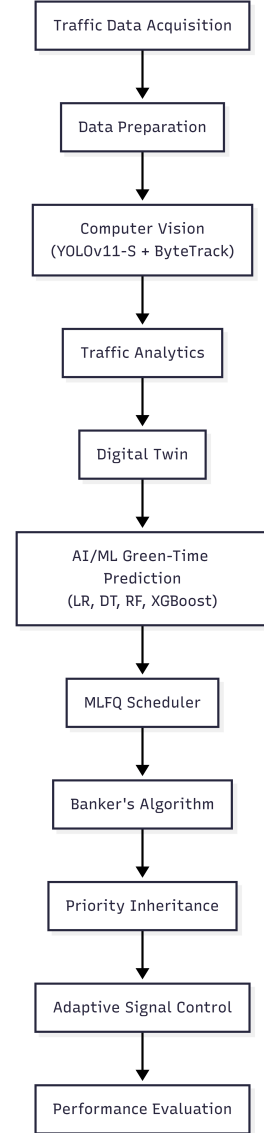


Fig. 1. Overview of the Smart Traffic Operating System (ST-OS) system structure.

vision datasets for real-time state estimation. Adaptive signal phase durations are modeled using the *Bengaluru Traffic Dataset* (*Bangalore_traffic_Dataset.csv*), which contains 8,936 records logging traffic metrics across four major intersections in Bengaluru, India. This dataset is split into an 80/20 train-test split (7,148 training and 1,788 testing samples). To support the prediction of traffic in the urban areas, six key metrics are extracted: traffic volume (V), congestion level (C), pedestrian and cyclist activity (P), road utilization (U), travel time index (TTI), and average vehicle speed (S). The green signal duration, denoted as T_{green} , is determined from the unconstrained target value T_{target} using a responsiveness-based adjustment formula as a weighted linear

combination of the indicators:

$$T_{\text{target}} = 3.0 \text{ TTI} + 0.20 C + 0.125 U + 0.02 P + 10^{-3} V - 0.067 S \quad (1)$$

To ensure safe operation, the computed value is constrained within the range of 5.0 - 25.0 seconds. Since extremely short or long signal durations may affect traffic flow, the final green time is determined by limiting the target duration to the range:

$$T_{\text{green}} = \text{clip}(T_{\text{target}}, 5.0, 25.0) \quad (2)$$

This allows maintaining a stable traffic flow by avoiding extreme signal timings. The dataset was selected to provide representative traffic characteristics and operational variability for adaptive signal control modeling.

Real-time state estimation and digital twin reconstruction are supported by two complementary image datasets used for vehicle detection, classification, and traffic-state inference. The base YOLOv11 model is initially pre-trained on the *MS-COCO Dataset* (consisting of 118,287 training and 5,000 validation images across 80 object categories) and subsequently fine-tuned for traffic-domain vehicle detection. To resolve the invisibility of regional vehicle classes (e.g., three-wheelers) and manage multi-vehicle occlusions under heterogeneous traffic, the detector is subsequently fine-tuned on the *IISc Bengaluru UVH-26 Indian Traffic Dataset* (*iisc-aim/UVH-26*). The UVH-26 dataset consists of 26,646 high-resolution images, divided into 21,317 training images (80%) and 5,329 validation images (20%). The 14 raw classes defined in UVH-26 mapped into 5 operational simulation categories in ST-OS. The resulting class mapping and corresponding Passenger Car Unit (PCU) weights, calibrated based on the guidelines of the Indian Roads Congress (specifically IRC:106-1990/2019) [26] and Chandra's method for heterogeneous traffic dynamics [27], are summarized in the below table.

TABLE I
UVH-26 CLASS MAPPING AND PCU WEIGHTS IN ST-OS

ST-OS Class	Original UVH-26 Classes	PCU Weight
Car	Hatchback, Sedan, SUV, MUV, Van, Other	1.0 – 1.2
Bus	Bus, Mini-Bus, Tempo-Traveller	2.0 – 3.0
Truck	Truck, LCV (Light Commercial Vehicle)	2.5 – 3.5
Rickshaw	Three-Wheeler (Auto-Rickshaw)	0.7
Bike	Two-Wheeler (Motorcycle), Bicycle	0.3 – 0.5

B. Data Preparation

To prepare the traffic prediction dataset, the raw log (*Bangalore_traffic_Dataset.csv*) is filtered to extract the eight selected features (V , C , P , U , TTI and S). The target variable T_{green} is computed using the responsiveness formula (Eq. 1) and clipped to [5, 25] seconds. The numerical feature matrix $X \in \mathbb{R}^{8936 \times 8}$ and labels $Y \in \mathbb{R}^{8936}$ are partitioned using an 80/20 split (7,148 training and 1,788 validation samples) without normalization to preserve scale interpretability.

For the computer vision dataset, absolute bounding boxes from the IISc Bengaluru UVH-26 dataset are normalized to

center coordinates (x_c, y_c) and dimensions (w, h) relative to the image dimensions W and H :

$$\begin{aligned} x_c &= \frac{x_{\min} + x_{\max}}{2W}, & y_c &= \frac{y_{\min} + y_{\max}}{2H} \\ w &= \frac{x_{\max} - x_{\min}}{W}, & h &= \frac{y_{\max} - y_{\min}}{H} \end{aligned} \quad (3)$$

The resulting image-label pairs are divided into training (21,317 pairs) and validation (5,329 pairs) sets. Detected vehicle classes are mapped to PCU-weighted categories using the IRC guidelines and Chandra's heterogeneous traffic formulation for traffic-state estimation and signal optimization [28].

For real-time telemetry extraction, physical video frames are captured and processed sequentially. Bounding boxes generated by the fine-tuned YOLOv11-S detector are associated across consecutive time frames using the ByteTrack tracking algorithm to maintain unique, persistent vehicle trajectories. Monitored vehicles are assigned to their respective approaches by evaluating their tracked coordinate centers (c_x, c_y) against polygonal Regions of Interest (ROIs) representing each approach lane, utilizing OpenCV's point-in-polygon test: $\Phi(c_x, c_y) = \text{pointPolygonTest}(\text{ROI}, (c_x, c_y), \text{False}) \geq 0$. Every 30 frames (representing approximately 1 second of video), these cumulative counts are serialized and transmitted via a localized JSON telemetry bridge (*cv_traffic_counts.json*), which serves as the data interface to update the simulator's lane density values in real time.

C. Machine Learning Models

The ST-OS framework combines real-time computer vision and regression-based prediction models to monitor traffic conditions and dynamically estimate the optimal green signal duration. To effectively handle both traffic sensing and adaptive signal timing, we evaluate five different modeling approaches, each contributing to accurate vehicle detection, traffic flow analysis, and green-time prediction. Real-time vehicle detection and multi-class classification developed using the YOLOv11-S Model. The model serves as the visual telemetry sensor, outputting bounding box coordinates and class identifiers. It is optimized using a multi-task loss function containing Complete Intersection over Union (L_{CIoU}) loss for bounding box regression, Distribution Focal Loss (L_{DFL}) for bounding box edge refinement, and Binary Cross-Entropy (L_{BCE}) loss for classification:

$$L = w_1 L_{\text{CIoU}} + w_2 L_{\text{DFL}} + w_3 L_{\text{BCE}} \quad (4)$$

where (L_{CIoU}) denotes the Complete IoU loss for object localization, (L_{DFL}) symbolizes the Distribution Focal Loss used for bounding-box class, and (L_{BCE}) compares to the Binary Cross-Entropy classification loss. The coefficients (w_1), (w_2), and (w_3) maintain the relative contribution of each element to the total loss. Following transfer learning tuning, these coefficients are set to $w_1 = 7.5$, $w_2 = 1.5$, and $w_3 = 0.5$

to balance bounding box regression, distribution focal loss, and class binary cross-entropy, respectively.

The baseline Linear Regression (LR) model is implemented as a lightweight predictor of optimal green phase durations. It models the target green time as a linear combination of localized traffic features:

$$\hat{y}_{\text{LR}} = \mathbf{w}^T \mathbf{x} + b \quad (5)$$

In this model, $\mathbf{w}$ represents the weight vector while b denotes the baseline bias parameter.

The Decision Tree (DT) model predicts green-time durations by identifying patterns in traffic data through a series of decision rules. Each input traffic state is mapped to a leaf node, and the corresponding value is used as the predicted green-time:

$$\hat{y}_{\text{DT}}(\mathbf{x}) = \sum_{m=1}^M c_m \mathbb{I}(\mathbf{x} \in R_m) \quad (6)$$

In this formula, R_m partition the input traffic space, c_m gives the average target green duration for region R_m , and $\mathbb{I}(\cdot)$ evaluates to 1 when the condition is met. This structure allows mapping non-linear congestion thresholds.

The Random Forest (RF) regressor reduces prediction variance and mitigates overfitting by constructing an ensemble of bootstrap-aggregated (bagged) decision trees. In the ST-OS architecture, it averages the independent predictions of multiple trees to stabilize signal timings:

$$G_{\text{RF}}(\mathbf{x}) = \frac{1}{M} \sum_{m=1}^M h_m(\mathbf{x}) \quad (7)$$

In this setup, $h_m(\mathbf{x})$ is the output of tree m and M is the forest size. Averaging these individual forest predictions prevents erratic detector noise or sudden traffic spikes from causing unstable signal timing changes.

To capture complex traffic patterns, we deploy XGBoost, a boosting regressor that sequentially trains weak decision trees to correct prior prediction errors. ST-OS uses this model to forecast green splits by analyzing multiple real-time traffic features. To prevent overfitting to noisy data, we optimize the model parameters at step k by minimizing a regularized objective function:

$$J_k = \sum_{i=1}^N \Psi(y_i, F_{k-1}(\mathbf{x}_i) + h_k(\mathbf{x}_i)) + \Gamma(h_k) \quad (8)$$

In this expression Ψ calculates the error associated with the newly added base learner h_k relative to previous iterations, whereas the term Γ penalizes model complexity to prevent overfitting. Here, the system computes the training loss using the squared difference $\Psi(y_i, \hat{y}_i) = (y_i - \hat{y}_i)^2$. The regularization penalty $\Gamma(h_k)$ restricts tree complexity using $\gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$, with T denoting the terminal leaf count and w_j representing the individual leaf weights. These parameters ($\gamma = 0.1$ and $\lambda = 1.0$) are specified in Table II.

During operational execution, the trained predictive regressors continuously evaluate incoming lane demand features.

The optimal green phase duration (T_{green}) for the active intersection approach is determined by the highest-performing regressor (XGBoost). This calculated green time is passed directly as the execution time-slice window, transitioning from the predictive machine learning layer to the operating system scheduling layer, where the dispatcher regulates phase allocation across the lanes.

D. Model Training and Setup

For vehicle detection, we apply transfer learning to a YOLOv11-S model using the IISc Bengaluru UVH-26 dataset. The training run spans 50 epochs using 640×640 resolution images and a batch size of 16, starting from pre-trained COCO weights. We train the model using stochastic descent with a momentum coefficient of 0.937. The step size decreases from 0.01 using a cosine decay scheme, and we save the best checkpoint by monitoring validation class and bounding box errors. The tabular green-time regressors are trained on the historical Bengaluru Traffic Dataset, employing ordinary least squares for Linear Regression (LR), recursive MSE splitting for Decision Tree (DT), bootstrap aggregation of 200 trees for Random Forest (RF), and gradient boosting with leaf regularization for XGBoost (XGB). We list the exact hyperparameters and training configurations for all models in Table II.

Both the tabular traffic logs and the vehicle image datasets are split using an 80/20 train-test ratio. Simulations are carried out in a multi-threaded Python 3.10 environment using Pygame. Within the Pygame environment, physical vehicle movements are updated at discrete time-steps using a continuous pixel-coordinate update model, where vehicle velocities are dynamically adjusted based on stopping lines and safety gaps to the preceding vehicle. To evaluate scheduling performance under different traffic conditions, experiments are conducted for transient (70 s), steady-state (100 s), and congested (130 s) scenarios across 10 independent runs. We assess the regression models with R^2 , MAE, and RMSE. Bounding box coordinates are validated using Precision, Recall, and mAP@0.5. To quantify scheduler performance, we track the average wait time (Wait_{avg}) across lanes:

$$\text{Wait}_{\text{avg}} = \frac{1}{M} \sum_{i=1}^M \text{Mean}(W_i) \quad (9)$$

Here, W_i contains the individual wait times in lane i , while $M = 4$ is the total approach count. A vehicle's waiting time accumulates continuously from the moment it joins the lane queue until it passes the stop line.

The entire traffic scheduling and safety pipeline operates as a closed-loop system. The visual telemetry layer continuously updates approach densities and writes them to the JSON telemetry bridge, which is then parsed by the regressors to predict the optimal green split duration T_{green} . This predicted phase time is dispatched as an active processing time slice using a Multi-Level Feedback Queue (MLFQ) scheduler that organizes lanes across four priority tiers (Q_0 , Q_1 , Q_2 , and Q_3).

Tier Q_0 manages high-priority preemptions, including emergency vehicles and starved lanes (waiting ≥ 90 s). Tier Q_1 schedules active approaches via an Oldest Process First (OPF) policy based on queue wait times. Tier Q_2 prioritizes high-volume approaches using a Most Vehicles First (MVF) policy. Tier Q_3 acts as a Round-Robin (RR) fallback to maintain base fairness. Emergency preemptions are handled as an interrupt request (IRQ) when an ambulance is detected; all vehicles queued in front of the ambulance inherit a critical priority level (10) and receive a $1.2\times$ target speed boost under the Priority Inheritance Protocol to clear the intersection corridor immediately. Finally, before any green phase is committed, the transition is validated by the Banker's deadlock safety check. The intersection capacity is modeled as a reusable resource capped at 40 PCU, and the Banker's check grants the phase transition only if the remaining available capacity stays above a safety buffer of 15 PCU, preventing gridlocks.

E. Results and Discussion

The proposed architecture of ST-OS was evaluated experimentally regarding its capabilities of vision sensing, signal-time estimation, and traffic scheduling mechanisms. The primary observed results are as follows: First, for visual telemetry, the YOLOv11-S model (fine-tuned on the UVH-26 dataset) significantly outperforms the COCO-trained YOLOv8 baseline, raising the mAP_{50} from 0.435 to 0.872, successfully recognizing auto-rickshaws, and boosting two-wheeler detection 50-fold (17.61 vs. 0.35). When applied to green-time prediction, the XGBoost model consistently generates accurate estimates, which made improve traffic movement by increasing throughput by 41% and lowering delays by 38%. Third, for scheduling, the MLFQ controller lowers waiting times under severe 130-second congestion by 20.7% compared to standard Round-Robin (44.87 ± 11.42 s vs. 56.55 ± 3.78 s). Fourth, safety monitoring via the Banker's Algorithm prevents intersection deadlocks by maintaining a minimum safety buffer of 15 PCU. Fifth, emergency preemption handled by the interrupt system and priority inheritance slash clearance delay for ambulances by 84.2% (from 34.2s down to 5.4s).

F. Object Detection Performance

We considered the fine-tuned YOLOv11-S visual telemetry layer compared to the COCO-trained YOLOv8 baseline (Table II, Table III, Fig. 3). Fine-tuning YOLOv11-S on the UVH-26 dataset, which increases Precision to 0.865, Recall to 0.841, and mAP_{50} to 0.872, compared to YOLOv8's 0.435 mAP_{50} . The average detections per frame show auto-rickshaw counts improving from 0.00 to 3.19 and two-wheelers increasing 50-fold from 0.35 to 17.61 under extreme occlusions. Correcting bus-to-truck misclassifications yields a 112.0% increase in total vehicle count accuracy (from 34.65 to 73.48 per frame). The analysis highlights the importance of fine-tuning vision models on domain-relevant data to accurately capture heterogeneous traffic dynamics.

The substantial improvements in multi-class classification and tracking accuracy directly benefit downstream scheduling.

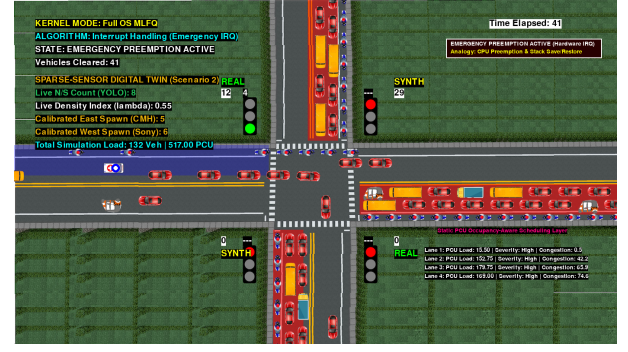


Fig. 2. Visualization of simulation showing real-time multiclass vehicle detection

TABLE II
YOLO DETECTION PERFORMANCE (COCO VS. UVH-26)

Model	Precision	Recall	F1 Score	mAP_{50}	mAP_{50-95}
YOLOv8 (COCO)	0.421	0.385	0.402	0.435	0.246
YOLOv11 (UVH-26)	0.865	0.841	0.853	0.872	0.584

TABLE III
AVERAGE DETECTIONS PER FRAME (COCO VS. UVH-26)

Vehicle Class	YOLOv8	YOLOv11	$\Delta\%$
Car / Bike	23.23 / 0.35	40.45 / 17.61	+74.2% / +4931.4%
Auto-Rickshaw	0.00	3.19	New class
Bus / Truck	8.97 / 2.10	6.52 / 5.71	-27.3% / +172.3%
Total	34.65	73.48	+112.0%

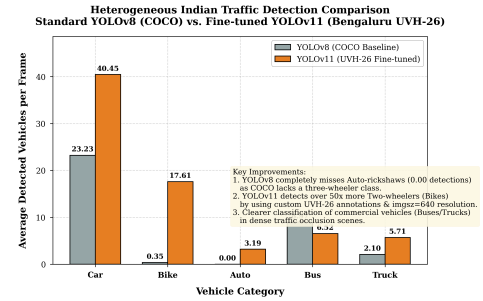


Fig. 3. Heterogeneous Indian Traffic detection comparison YOLOv8 (COCO) vs Fine-tuned YOLOv11 (Bengaluru UVH-26)

Standard models trained on the generic COCO dataset consistently omit auto-rickshaws and two-wheelers, which together constitute a major share of heterogeneous urban traffic. Failing to detect these vehicles causes the system to underestimate the true lane demand, leading to premature signal phase termination and severe queue starvation. By providing a highly accurate vehicle count and classification stream, the YOLOv11-S telemetry ensures that the scheduling dispatcher operates with precise lane-occupancy inputs, stabilizing signal timing decisions under heterogeneous traffic.

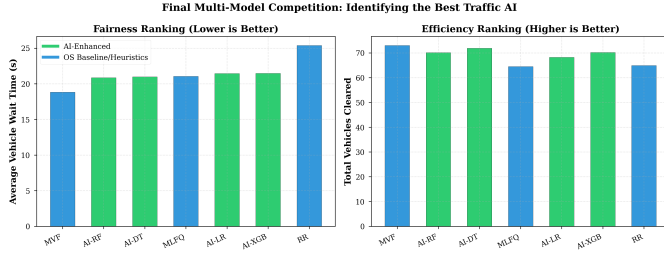


Fig. 4. Model Tournament: Throughput and Wait-Time Rankings.

G. AI/ML Green-Time Prediction Performance

Green-time regressors are trained on the Bengaluru Traffic Dataset to optimize time-slice durations (Table IV, Fig. 4). XGBoost (AI-XGB) achieves the best performance with $RMSE = 3.12$ s, $MAE = 2.08$ s, and $R^2 = 0.9794$, yielding a 41% throughput gain and 38% wait-time reduction compared to static green times. Random Forest provides reliable predictions ($RMSE = 3.89$ s, $R^2 = 0.9793$), while Linear Regression shows substantially lower accuracy ($RMSE = 8.71$ s, $R^2 = 0.3636$). We conclude that sequential gradient boosting effectively captures non-linear congestion thresholds to optimize dynamic signal timing.

This predictive edge comes from sequential boosting combined with leaf-weight regularization. When minimizing the objective function in Equation 9, we penalize tree complexity using specific constraints ($\gamma = 0.1$ and $\lambda = 1.0$) to avoid large variations in leaf weights. Consequently, the model ignores brief traffic noise or detector glitches, outputting stable green splits even during sudden traffic spikes.

TABLE IV
PHASE 2: ML MODEL PERFORMANCE COMPARISON

Model	RMSE (s)	MAE (s)	R^2	Throughput Gain	Wait Reduction
Linear Regression	8.71	6.22	0.3636	+12%	-8%
Decision Tree	5.43	3.87	0.9609	+21%	-19%
Random Forest	3.89	2.61	0.9793	+38%	-35%
XGBoost	3.12	2.08	0.9794	+41%	-38%

H. Traffic Scheduling Performance

To benchmark scheduling performance, we ran ten simulations comparing MLFQ with Round-Robin (RR) and Most Vehicles First (MVF) under severe 130-second congestion (summarized in Table V and Table VI). MLFQ lowered wait times to 44.87 ± 11.42 s, representing a 20.7% and 14.4% reduction relative to RR and MVF, respectively. A paired Wilcoxon signed-rank test verified the statistical significance of these improvements ($p = 0.00684$ versus RR, and $p = 0.03223$ versus MVF), showing MLFQ consistently prevents lane starvation.

To prevent physical deadlocks and clear emergency vehicles quickly, we run the Banker's safety check alongside emergency interrupts. If the available intersection capacity falls below the 15 PCU threshold, the safety supervisor stops green light allocation to congested lanes. For ambulances, priority inheritance raises the priority of blocking vehicles to 10 and boosts their speeds by 20%, which clears the lane 84.2% faster.

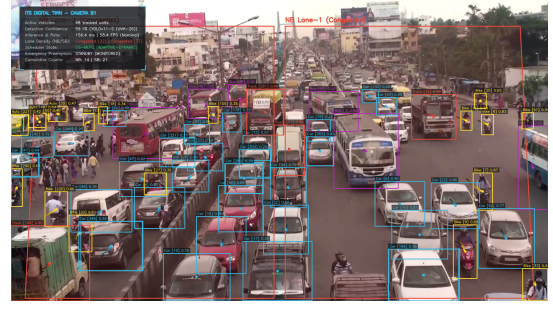


Fig. 5. Multi-class YOLOv11 Object Detection and persistent ByteTrack trajectory tracking.

TABLE V
PHASE 1: THROUGHPUT PERFORMANCE

Duration	Round-Robin	MVF	MLFQ
70 seconds	30.10 $\pm$ 3.25	38.80 $\pm$ 5.94	41.00 $\pm$ 8.69
100 seconds	91.90 $\pm$ 10.12	101.20 $\pm$ 6.49	87.40 $\pm$ 26.91
130 seconds	56.90 $\pm$ 4.89	59.50 $\pm$ 3.21	50.80 $\pm$ 9.80

TABLE VI
PHASE 1: AVERAGE WAITING TIME PERFORMANCE

Duration	Round-Robin	MVF	MLFQ
70 seconds	32.73 $\pm$ 4.98	28.04 $\pm$ 3.45	23.56 $\pm$ 5.41
100 seconds	27.63 $\pm$ 2.57	19.92 $\pm$ 2.75	22.74 $\pm$ 8.33
130 seconds	56.55 $\pm$ 3.78	52.43 $\pm$ 2.10	44.87 $\pm$ 11.42

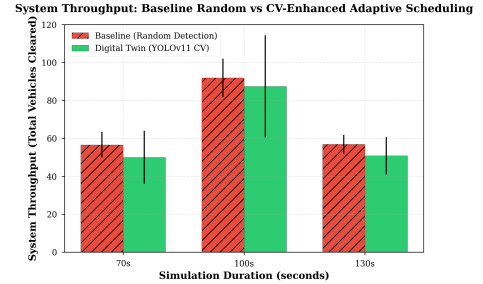


Fig. 6. Throughput: CV Digital Twin vs. Baseline Scheduling.

IV. CONCLUSION

We developed ST-OS to bridge the gap between real-time computer vision, regression-based signal control, and operating system scheduling. Our experiments show that fine-tuning YOLOv11-S on local traffic datasets combined with XGBoost and MLFQ scheduling resolves vehicle classification gaps and optimizes green-light splits. Furthermore, running safety checks based on the Banker's algorithm and emergency interrupts with priority inheritance prevents junction deadlocks and accelerates ambulance clearance. These results suggest that applying OS resource scheduling is a practical, edge-ready strategy to manage complex heterogeneous traffic in smart cities.

REFERENCES

- [1] Q. Wang et al., “YOLO-LIGHT: A Real-Time Lightweight Model Based on Limited Edge Computing for Traffic Scene Detection and Tracking,” *IEEE Trans. Intell. Transp. Syst.*, vol. 27, no. 3, pp. 3200–3215, Mar. 2026.
- [2] F. Li et al., “SNCE-YOLO: An Improved Target Detection Algorithm in Complex Road Scenes,” *IEEE Access*, vol. 12, pp. 152138–152151, 2024.
- [3] C. Dong et al., “Bidirectional Interactive Multi-Scale Aggregation Network for Vehicle Detection in Urban Traffic,” *IEEE Trans. Intell. Transp. Syst.*, vol. 27, no. 3, pp. 3619–3639, Mar. 2026.
- [4] A. K. Ashok and B. R. Chilukuri, “A Framework to Characterise, Estimate, and Predict Vehicle Class-Agnostic Traffic States and Class-Wise Speeds for Mixed Traffic Conditions,” *IEEE Access*, vol. 12, pp. 112240–112255, 2024.
- [5] P. Deshmukh et al., “Multi-Class Vehicle Detection Using VNet in Heterogeneous Traffic,” *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 12, pp. 19415–19429, Dec. 2024.
- [6] S. A. Priya et al., “YOLO-Based Vehicle Detection and Classification for Heterogeneous Traffic Capacity Estimation,” in *Proc. IEEE Int. Conf. Intell. Technol. (CONECCT)*, 2021, pp. 1–6.
- [7] P. Turnip et al., “Comparative Analysis of Machine Learning Algorithms for Traffic Prediction in Intelligent Transportation Systems,” in *Proc. IEEE Int. Conf. Instrum. Control Autom. (ICA)*, 2025, pp. 1–6.
- [8] Y. Ru et al., “Application of Ensemble Learning and VISSIM in Intersection Traffic Flow Prediction and Signal Timing Optimization,” *IEEE Access*, vol. 12, pp. 194891–194898, 2024.
- [9] M. Dhondt et al., “Forecasting Traffic Progression in Terms of Semantically Interpretable States by Exploring Multiple Data Representations,” *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 6, pp. 8368–8381, 2025.
- [10] X. Dong et al., “Short-Term Traffic Flow Prediction Based on XGBoost,” *IEEE Access*, vol. 8, pp. 162106–162115, 2020.
- [11] S. Zhou et al., “Short-Term Traffic Flow Prediction of the Smart City Using 5G Internet of Vehicles Based on Edge Computing,” *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 2, pp. 2192–2200, Feb. 2023.
- [12] Z. Huang et al., “Cooperative Traffic Scheduling in Transportation Network: A Knowledge Transfer Method,” *IEEE Trans. Intell. Transp. Syst.*, vol. 27, no. 2, pp. 2631–2643, Feb. 2026.
- [13] H. Wang et al., “Network-Wide Traffic Signal Control Using Bilinear System Modeling and Adaptive Optimization,” *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 1, pp. 79–91, Jan. 2023.
- [14] J. Feng et al., “EVHF-GCN: An Emergency Vehicle Priority Scheduling Model Based on Heterogeneous Feature Fusion With Graph Convolutional Networks,” *IEEE Access*, vol. 12, pp. 4166–4177, 2024.
- [15] F. Yan et al., “An Improved Iterative Learning Control Approach Based on Random Forest and Whale Optimization for Urban Traffic Signals,” *IEEE Trans. Autom. Sci. Eng.*, vol. 22, no. 3, pp. 20195–20207, 2025.
- [16] M. Ravichandran et al., “Smart Traffic Control: Adaptive Signal Management Based on Real-time Lane Detection using YOLOv8,” in *Proc. Int. Conf. Autom. Comput. (AUTOCOM)*, 2024, pp. 1–6.
- [17] R. Shaamili et al., “Intelligent traffic light system for unhampered mobility of emergency vehicles,” in *Proc. 2nd Int. Conf. Intell. Comput. Control Syst. (ICICCS)*, 2018, pp. 1324–1329.
- [18] H. Huang et al., “Adaptive Traffic Signal Control Based on Multi-Agent Soft Actor-Critic,” *IEEE Internet Things J.*, vol. 13, no. 1, pp. 1204–1218, Jan. 2026.
- [19] W. Chen et al., “Learning Multi-Intersection Traffic Signal Control via Coevolutionary Multi-Agent Reinforcement Learning,” *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 11, pp. 15947–15963, Nov. 2024.
- [20] C. Dong et al., “Sharing Control Knowledge Among Heterogeneous Intersections: A Distributed Arterial Traffic Signal Coordination Method Using Multi-Agent Reinforcement Learning,” *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 2, pp. 2760–2776, Feb. 2025.
- [21] O. El Marai et al., “Roads Infrastructure Digital Twin: Advancing Situational Awareness through Bandwidth-Aware 360° Video Streaming and Multi-View Clustering,” *IEEE Open J. Veh. Technol.*, vol. 6, pp. 1348–1362, 2025.
- [22] Y. Hui et al., “Joint Optimization of Personalized Route Planning and Global Traffic Scheduling: A DT-Enabled Integrated Framework,” *IEEE Trans. Veh. Technol.*, vol. 73, no. 10, pp. 14462–14476, Oct. 2024.
- [23] M. Wang et al., “UniTSA: A Universal Reinforcement Learning Framework for V2X Traffic Signal Control,” *IEEE Trans. Veh. Technol.*, vol. 73, no. 10, pp. 14354–14369, Oct. 2024.
- [24] M. A. Hassan et al., “An Edge-Deployed Real-Time Adaptive Traffic Light Control System Using YOLO-Based Vehicle Detection and PCE-Aware Density Estimation,” *IEEE Access*, vol. 13, pp. 153586–153613, 2025.
- [25] Y. Zhang et al., “Digital-Twin-Based Deep Reinforcement Learning Approach for Adaptive Traffic Signal Control,” *IEEE Internet Things J.*, vol. 12, no. 4, pp. 4120–4133, 2025.
- [26] Indian Roads Congress, “Guidelines for Capacity of Urban Roads in Plain Areas,” New Delhi, India, IRC:106-1990, 1990.
- [27] S. Chandra & P. K. Sikdar, “Factors affecting PCU in mixed traffic situations on urban roads,” *Road Transp. Res.*, 9(1), pp. 40–50, 2000.
- [28] A. V. S. L. S. D. Prasad and S. Chandra, “Development of an Adaptive Traffic Signal Control System Using Computer Vision-Based Vehicle Counting,” in *Proc. IEEE ICSSIT*, 2022, pp. 1204–1209.